

Unit 1. Introduction to Java

History of Java:

- James Gosling initiated the Java language project in June 1991 for use in one of his many set-top box projects. The language, initially called Oak after an oak tree that stood outside Gosling's office, also went by the name Green and ended up later renamed as Java, from a list of random words.
- Sun released the first public implementation as Java 1.0 in 1995. It promised **Write Once, Run Anywhere** (WORA), providing no-cost run-times on popular platforms.
- On 13 November 2006, Sun released much of Java as free and open source software under the terms of the GNU General Public License (GPL).
- On 8 May 2007 Sun finished the process, making all of Java's core code free and open-source, aside from a small portion of code to which Sun did not hold the copyright.

Comparison in Java and C++

Java	C++
Java is true Object-oriented language.	C++ is basically C with Object-oriented extension.
Java does not support operator overloading.	C++ supports operator overloading.
It supports labels with loops and statement blocks	It supports goto statement.
Java does not have template classes as in C++.	C++ has template classes.
Java compiled into byte code for the Java Virtual Machine. The source code is independent on operating system.	Source code can be written to be platform independent and written to take advantage of platform.C++ typically compiled into machine code.
Java does not support multiple inheritance of classes but it supports interface.	C++ supports multiple inheritance of classes.
Runs in a protected virtual machine.	Exposes low-level system facilities.
Java does not support global variable. Every variable should declare in class.	C++ support global variable.
Java does not use pointer.	C++ uses pointer.
It Strictly enforces an object oriented programming paradigm.	It Allows both procedural programming and object-oriented programming.
There are no header files in Java.	We have to use header file in C++.

JAVA Features:

- The Java is an object oriented programming language developed by Sun Microsystems of USA in 1991.
- Program developed in Java can be executed anywhere and on any system.

Features of java:

1. Compiled and Interpreted

- Basically a computer language is either compiled or interpreted. Java comes together both these approach thus making Java a two-stage system.
- Java compiler translates Java code to Bytecode instructions and Java Interpreter generate machine code that can be directly executed by machine that is running the Java program.

2. Platform Independent

- Java is Platform independent and supports the feature portability.
- Java programs can be easily moved from one computer system to another and anywhere.
- Changes and upgrades in operating systems, processors and system resources will not force any alteration in Java programs.

Java certifies **portability** in two ways.

First way:=

- Java compiler generates the **bytecode**.the that can be executed on any machine.
- the Java interpreter(Java Virtual Machine) reads the byte code, translates it into the corresponding host machine instructions and then executes the machine instruction.
- A Java program can run on any computer system for which a JVM (Java Virtual Machine) and some library routines have been installed.

Second way :=

- size of primitive data types are machine independent.
- Integers are always four bytes long and floating-point variables follow the IEEE 754.You don't need to worry that the interpretation of your integer is going to change if you move from one hardware to another hardware like Pentium to a PowerPC.
- the execution of that program is possible on any other computer system loaded with JVM.
- For example, you can write and compile the Java program on Windows 98 and execute the compiled program on JVM of the Macintosh operating system.

The same concept is explained in Figure 1given below.

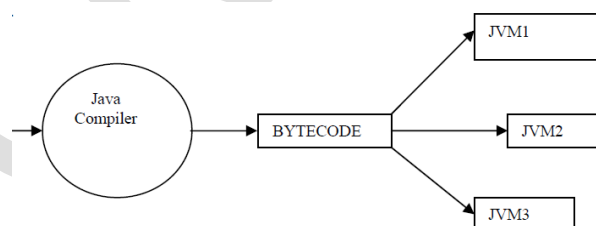


Figure 1: Compilation and execution of Java program

3. Object

Oriented

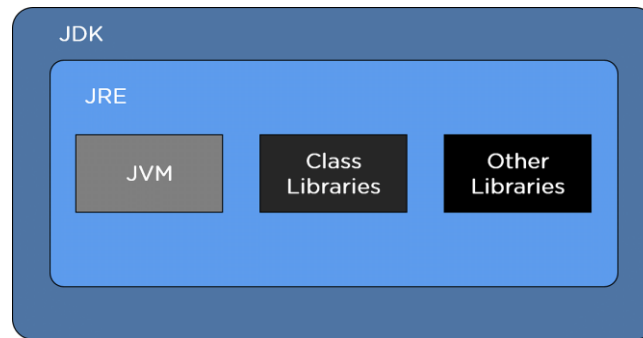
- Java is truly object-oriented language.
- Objects have two sections.
- The first is Data (instance variables) and the second is methods.
 - Data represents what an object is. A method represents what an object does.
- In Java, almost everything is an Object. The object model in Java is trouble-free,easy to enlarge, reuse of programming segments and robust.

4. Easy to Learn

- Java is very small and simple language. Java does not use **pointer** and **header files, goto statements, etc.**
 - It eliminates **operator overloading , multiple inheritance and pointers.**
 - in Java is **automatic memory allocation** and **de-allocation.**
5. **Robust and secure**
- Java is a most strong language which provides many securities to make certain reliable code.
 - many times **pointers** are the main cause of **runtime errors** because of improper use of memory. Java eliminates pointer manipulation completely from the language, and therefore eliminates a large source of runtime errors. Java programmers need not remember to de-allocate memory in programs since there is a **garbage collection mechanism** which handles de-allocation of memory. It provides powerful a robust exception handling mechanism to deal with both expected and unexpected errors at run time.
 - Java also includes the concept of exception handling, which control serious errors and reduces all kind of threat of crashing the system.
 - Security is an important feature of Java and this is the strong reason that programmer use this language for programming on Internet.
6. **Multithreaded and Interactive**
- The execution of two or more sections of a program at the same time and this technique is known as multithreading.
 - Multithreaded means managing multiple tasks simultaneously. Java maintains multithreaded programs. That means we need not wait for the application to complete one task before starting next task. This feature is helpful for graphic applications
7. **Dynamic and Extensible**
- The Java compiler is smart and dynamic.
 - If you are compiling a file that depends on other non-compiled files, then the compiler will try to find and compile them also.
 - The compiler can handle methods that are used before they're declared. It can also determine whether a source code has been changed since the last time it was compiled.
8. **High Performance**
- Java is a fast-interpreted language.
 - Java has designed for the run-time system can optimize their performance by compiling bytecode to native machine code on the fly (execute immediately after compilation). This is called "just in time" (JIT) compilation.

JDK

JDK in Java is an abbreviation for Java Development Kit. It is a bundle of **software development tools and supporting libraries combined** with the Java Runtime Environment (JRE) and Java Virtual Machine (JVM).



Jdk=jre+development tool kit(javac,Javadoc...)

Jre=jvm+java class+run time library

- Java Development kit (JDK) – The JDK comes with a set of tools that are used for developing and running Java program. It includes:
 1. Appletviewer(It is used for viewing the applet)
 2. Javac(It is a Java Compiler)
 3. Java(It is a java interpreter)
 4. Javap(Java diassembler,which convert byte code into program description)
 5. Javah(It is for java C header files)
 6. Javadoc(It is for creating HTML document)
 7. Jdb(It is Java debugger)

Java runtime environment (JRE)

Java Development Kit (JDK) and Java Runtime Environment (JRE) **both interact** with each other to create a runtime environment.

JRE has an object of **JVM with it, development tools, and library classes**. To understand the working of Java Runtime Environment

The JRE runtime architecture consists of **ClassLoader, ByteCode verifier and Interpreter**.

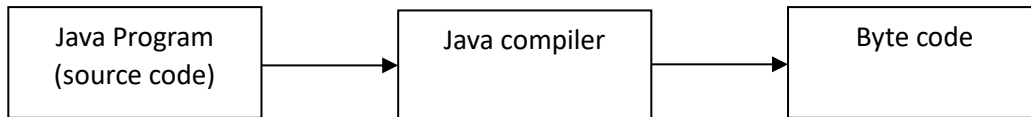
- Java ClassLoader dynamically **loads all the classes** necessary to run a Java program. For example System class loader.
- The bytecode checker ensures the **format and accuracy** of Java code before passing it to the interpreter.
- After loading the byte code successfully, the Java interpreter creates an object of the Java virtual machine that allows the Java program **to run** natively on the underlying machine.

Java Virtual machine:

- As we know that all programming language **compilers** convert the **source code** to **machine code**.
- In java, **Java compiler** convert the **source code** into **Intermediate code** is called as **bytecode**.

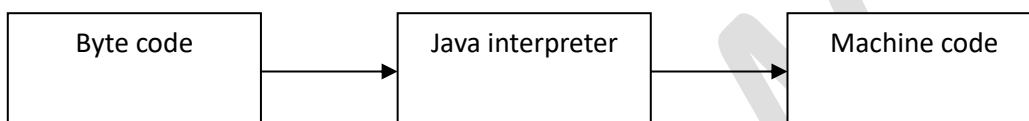
- This machine is called the **Java Virtual machine** and it exists only inside the computer memory.

the process of compilation



- The byte code is not machine specific.
- The **machine code** is generated By **Java interpreter** by acting as an **intermediary** between the two different machines shown below

the process of running program



A Simple Java Program

Java offers two flavors of programming, Java applets(embedded in the webpage) and Java application

Let us write your first program with the file name “Application”. This program, after successful compilation and run, prints the words “This is My First Java Program” on your display.

1. First of all using any text editor, type Java source file of your program as given below.
2. Now compile the source file-using compiler named **Javac** that takes your **source file** and translates its statements into a **bytecode** file.
3. Run the program using interpreter named Java that takes your **bytecode** file and translates them into **machine code** that your computer can understand.

```

/* This is my First Java Application Save the file as Application.Java: same as class name */
class Application
{
    public static void main (String args[ ] )
    {
        System.out.println("This is My First Java Application");
    }
    // main ends here
} // Code ends here
  
```

Let us see the above program line by line.

Line 1. /* This is my First Java Program. Save this file as Application.Java same as class name */

Comments in Java are similar to C++. Everything between `/*` and `*/` and `//` is ignored by the compiler but Comments allow you to describe the details of the program. This is useful for developing the understandability in your program.

Line 2. class Application

Second line in the program defines a class named Application using a keyword class.

Line 4. `public static void main (String args[ ])/static public void main(String []args)/public static void main(String[] arg)/public static void main(String... arg)`

This is the point from where the program will start the execution. This program **starts the execution** by calling **main ()** method. In this line public, static, and void all are keywords. May be you are thinking of the meaning of 'keyword'. Keywords are nothing but some reserved words.

- The **public** keyword is used to control the access of various class members. If member is public it can be **accessed outside the class**.
- So we have to declare main () as public because it has to be **invoked by the code outside the class** when program is executed.
- **Static** key word allows this function **remains in memory** throughout the execution of the application
- and **void** means main () method does **not return any value**.
- **Parentheses ()**

It encloses **arguments** in method definitions or calling and used to define test expressions control statements.

- **args** can be used in the main function to pass parameters from the operating system command line
- **Braces { }**
It defines blocks of code
- **Square bracket []**
It declares array types.
- **Semicolons;**
are used to terminate statements.
- **Single dot "."**

It Selects a method from an object and separates package names from subpackage and class names.

Line 6. `System.out.println("This is My First Java Application.");`

This line prints the string "This is My First Java Application".

- **System** is class
- **out/in/err** is variable of printstream which is present in System class. `Println(print in next line print)/print/printf(formatted)` is method or printstream class method.

This function is used to display this line. The `println ()` function accepts any string and display its value on console.

Compiling the Program

To compile the Example program, execute the compiler, `javac`, specifying the name of the source file on the command line, as shown here:

C:\>javac Application.java

- The javac compiler creates a file called Example.class that contains the bytecode version of the program.
- The Java bytecode is the intermediate of program.the Java interpreter will execute.
- Thus, the output of javac is not code that can be directly executed.
- To actually run the program, you must use the Java interpreter, called java. To do so, pass the class name Example as a command-line argument, as shown here:

C:\>java Application

Output:

This is My First Java Application.

Identifier Literals, Operators, Variables, Keywords, Data Types

identifier

Identifiers are the words a programmer uses in a program. Most identifiers have no predefined meaning except as specified by the programmer

Rules...

- An identifier can be made up of letters, digits, the underscore character (_), and the dollar sign (\$)
- The first character must be any non-digit
- Subsequent characters may include digits
- Avoid using underscore and \$ for the first character
- Java is case sensitive, therefore Total and total are different identifiers

Literals

Literals are data used for representing fixed values. They can be used directly in the code. For example,

```
int a = 1;
float b = 2.5;
char c = 'F';
```

Types of literals

1. Boolean Literals

In Java, boolean literals are used to initialize boolean data types. They can store two values: true and false. For example,

```
boolean flag1 = false;
boolean flag2 = true;
```

2. Integer Literals: An integer literal is a numeric value(associated with numbers) without any fractional or exponential part. There are 4 types of integer literals in Java:

1. binary (base 2)
2. decimal (base 10)
3. octal (base 8)
4. hexadecimal (base 16)

For example:

```
int octalNumber = 027;
```

```
int decNumber = 34;
int hexNumber = 0x2F; // 0x represents hexadecimal
int binNumber = 0b10010; // 0b represents binary
```

In Java, binary starts with **0b**, octal starts with **0**, and hexadecimal starts with **0x**.

3. Floating-point Literals: A floating-point literal is a numeric literal that has either a fractional form or an exponential form. For example,

```
double myDouble = 3.4;
float myFloat = 3.4F;
double myDoubleScientific = 3.445e2;
```

4. Character Literals: Character literals are unicode character enclosed inside single quotes. For example,

```
char letter = 'a';
```

5. String literals: A string literal is a sequence of characters enclosed inside double-quotes. For example,

```
String str1 = "raman";
```

Operators

Operators are symbols that perform operations on variables and values. Operators in Java can be classified into 5 types:

1. Arithmetic Operators
2. Assignment Operators
3. Relational Operators
4. Logical Operators
5. Unary Operators
6. Bitwise Operators

1. Arithmetic Operators: Arithmetic operators are used to perform arithmetic operations on variables and data.

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo Operation (Remainder after division)

2. Assignment Operators: Assignment operators are used in Java to assign values to variables.

Operator	Example	Equivalent to
=	a = b;	a = b;
+=	a += b;	a = a + b;
-=	a -= b;	a = a - b;
*=	a *= b;	a = a * b;
/=	a /= b;	a = a / b;
%=	a %= b;	a = a % b;

3. Relational Operators: Relational operators are used to check the relationship between two operands.

Operator	Description	Example
==	Is Equal To	3 == 5 returns false
!=	Not Equal To	3 != 5 returns true

>	Greater Than	3 > 5 returns false
<	Less Than	3 < 5 returns true
>=	Greater Than or Equal To	3 >= 5 returns false
<=	Less Than or Equal To	3 <= 5 returns true

4. Logical Operators: Logical operators are used to check whether an expression is `true` or `false`

Operator	Example	Meaning
&& (Logical AND)	expression1 && expression2	true only if both expression1 and expression2 are true
(Logical OR)	expression1 expression2	true if either expression1 or expression2 is true
! (Logical NOT)	!expression	true if expression is false and vice versa

5. Unary Operators: Unary operators are used with only one operand. For example, `++` is a unary operator that increases the value of a variable by `1`. That is, `++5` will return `6`.

Operator	Meaning
+	Unary plus: not necessary to use since numbers are positive without using it
-	Unary minus: inverts the sign of an expression
++	Increment operator: increments value by 1
--	Decrement operator: decrements value by 1
!	Logical complement operator: inverts the value of a Boolean

6.Bitwise Operators: Bitwise operators in Java are used to perform operations on individual bits.

Operator	Description
~	Bitwise Complement
<<	Left Shift
>>	Right Shift
&	Bitwise AND
^	Bitwise exclusive OR

Ternary Operator

The ternary operator (conditional operator) is shorthand for the `if-then-else` statement.

Syntax:

variable = Expression ? expression1 : expression2

Here's how it works.

- If the `Expression` is `true`, `expression1` is assigned to the `variable`.
- If the `Expression` is `false`, `expression2` is assigned to the `variable`.

Example:

```
int a=2;
int b=5;
int min=(a<b)?a:b;
System.out.println(min);
```

Output:

2

Variables

A variable is a location in memory (storage area) to hold data.

To indicate the storage area, each variable should be given a unique name

Create Variables in Java

```
int speedLimit = 80;
```

Change values of variables

```
int speedLimit = 80;
```

```
... ..
```

```
speedLimit = 90;
```

Keywords

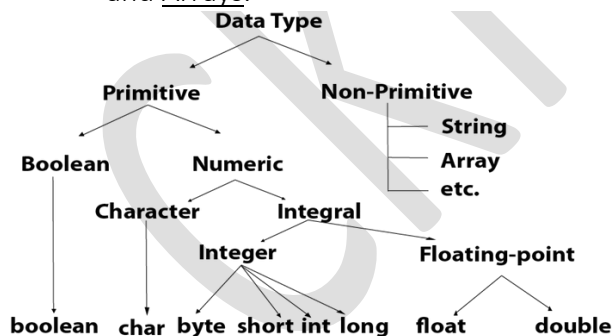
Keywords are predefined, reserved words used in Java

Java Keywords List				
Abstract	assert	boolean	break	byte
Case	catch	char	class	const
Continue	default	do	double	else
Enum	extends	final	finally	Float
For	goto	if	implements	Import
Instanceof	int	interface	long	Native
New	package	private	protected	Public
Return	short	static	strictfp	Super
Switch	synchronized	this	throw	Throws
Transient	try	void	volatile	While

Data Types

Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:

1. **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
2. **Non-primitive data types:** The non-primitive data types include Classes, Interfaces, and Arrays.



Data Type	Default Value	Default size
boolean	false	1 bit
char	'\u0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte
long	0L	8 byte
Float	0.0f	4 byte
double	0.0d	8 byte

Selection statements:

1. if statement

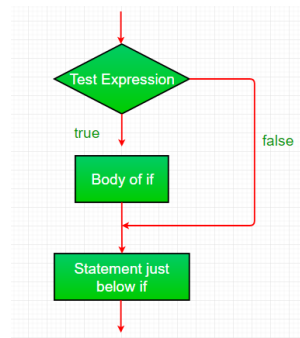
if statement is the most simple decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e if a certain condition is true then a block of statements is executed otherwise not.

Syntax:

```
if(condition)
{
    // Statements to execute if
    // condition is true
}
```

Here, the **condition** after evaluation will be either true or false. If we do not provide the curly braces '{' and '}' after **if(condition)** then by default if statement will consider the immediate one statement to be inside its block.

Flowchart:



Example:

```
Public class main
{
    public static void main(String args[])
    {
        int i = 10;
        if (i < 15)
            System.out.println("10 is less than 15");
    }
}
```

Output

10 is less than 15

2. if-else statements

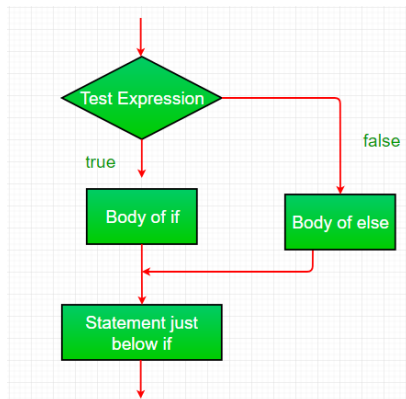
- The **if** statement executes a certain section of code if the test expression is evaluated to **true**. However, if the test expression is evaluated to **false**, it does nothing.
- In this case, we can use an optional **else** block. Statements inside the body of **else** block are executed if the test expression is evaluated to **false**. This is known as the **if-...else** statement in Java.

Syntax:

```
if (condition) {
    // codes in if block
}
```

```
else {
    // codes in else block
}
```

Flowchart:



Example:

```
class main {
    public static void main(String args[])
    {
        int i = 10;
        if (i < 15)
            System.out.println("i is smaller than 15");
        else
            System.out.println("i is greater than 15");
    }
}
```

Output

i is smaller than 15

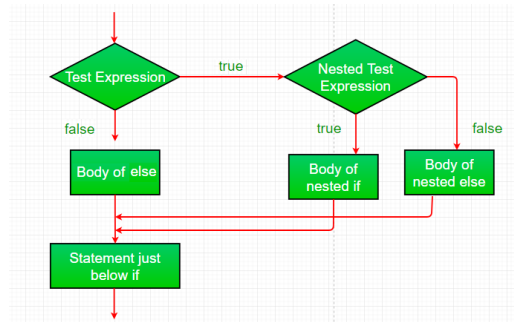
3. nested-if statements

nested-if: A nested if is an if statement that is the target of another if or else. Nested if statements mean an if statement inside an if statement. Yes, java allows us to nest if statements within if statements. i.e, we can place an if statement inside another if statement.

Syntax:

```
if (condition1)
{
    // Executes when condition1 is true
    if (condition2)
    {
        // Executes when condition2 is true
    }
}
```

Flowchart:



Example:

```

class main {
    public static void main(String args[])
    {
        int i = 10;
        if (i == 10 || i < 15) {
            // First if statement
            if (i < 15)
                System.out.println("i is smaller than 15");
            // Nested - if statement
            // Will only be executed if statement above
            // it is true
            if (i < 12)
                System.out.println(
                    "i is smaller than 12 too");
            } else{
                System.out.println("i is greater than 15");
            }
        }
    }
}
  
```

Output

```

i is smaller than 15
i is smaller than 12 too
  
```

4. if-else-if statment

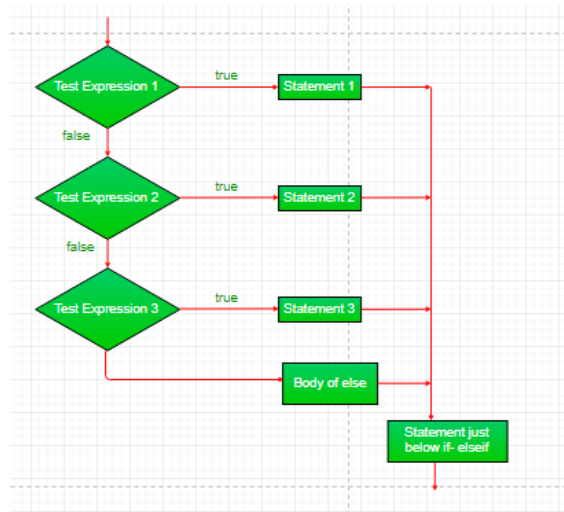
It is use for multiple options. The if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that 'if' is executed, and the rest of the ladder is bypassed. If none of the conditions is true, then the final else statement will be executed. There can be as many as 'else if' blocks associated with one 'if' block but only one 'else' block is allowed with one 'if' block.

Syntax:

```

if (condition)
    statement;
else if (condition)
    statement;
.
.
else
    statement;
  
```

Flowchart:



Example:

```

class main {
    public static void main(String args[])
    {
        int i = 20;

        if (i == 10)
            System.out.println("i is 10");
        else if (i == 15)
            System.out.println("i is 15");
        else if (i == 20)
            System.out.println("i is 20");
        else
            System.out.println("i is not present");
    }
}
  
```

Output

i is 20

5. [switch-case](#)

The switch statement is a multiway branch statement. It provides an easy way to dispatch execution to different parts of code based on the value of the expression.

Syntax:

```

switch (expression)
{
    case value1:
        statement1;
        break;
    case value2:
        statement2;
        break;
    .
    .
    case valueN:
        statementN;
}
  
```

```

        break;
    default:
        statementDefault;
}

```

Example:

```

Class main {
    public static void main (String[] args) {
        int num=20;
        switch(num){
            case 5 : System.out.println("It is 5");
                break;
            case 10 : System.out.println("It is 10");
                break;
            case 15 : System.out.println("It is 15");
                break;
            case 20 : System.out.println("It is 20");
                break;
            default: System.out.println("Not present");
        }
    }
}

```

Output

It is 20

6. jump – break, continue, return

Java supports three jump statements: **break**, **continue** and **return**. These three statements transfer control to another part of the program.

Break: In Java, a break is majorly used for:

- Terminate a sequence in a switch statement (discussed above).
- To exit a loop.
- Used as a “civilized” form of goto.

Example:

```

class main {
    public static void main(String args[])
    {
        for (int i = 1; i < 10; i++) {
            // If the number is even
            // break
            if (i == 5)
                break;

            // If number is 5
            System.out.print(i + " ");
        }
    }
}

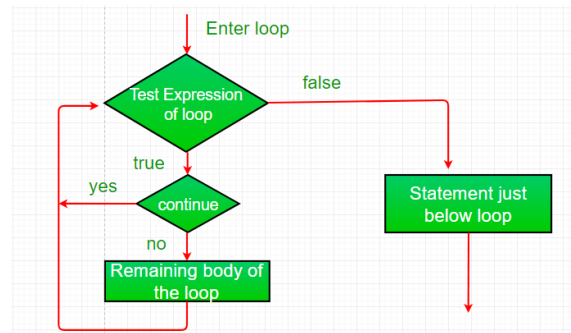
```

Output

1 2 3 4

Continue: Sometimes it is useful to force an early iteration of a loop. That is, you might want to continue running the loop but stop processing the remainder of the code in its body for this particular iteration. This is, in effect, a goto just past the body of the loop, to the loop's end. The continue statement performs such an action.

Flowchart:



Example:

```

class main {
    public static void main(String args[])
    {
        for (int i = 0; i < 10; i++) {
            // If the number is even
            // skip and continue
            if (i % 2 == 0)
                continue;
            // If number is odd, print it
            System.out.print(i + " ");
        }
    }
}
  
```

Output

1 3 5 7 9

Return: The return statement is used to explicitly return from a method. That is, it causes program control to transfer back to the caller of the method.

Example:

```

class Return {
    public static void main(String args[])
    {
        boolean t = true;
        System.out.println("Before the return.");

        if (t)
            return;

        // Compiler will bypass every statement
        // after return
        System.out.println("This won't execute.");
    }
}
  
```



```

    }
}

```

Output

Before the return.

Note: "Return" terminates the entire function, while "break" only exits the loop

Looping: While, Do-while, For

1. while loop:

A while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition. The while loop can be thought of as a repeating if statement.

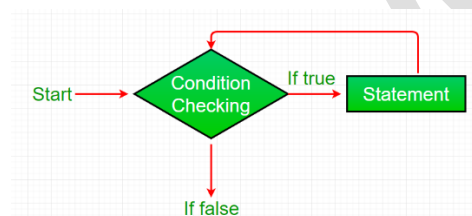
Syntax :

```

while (boolean condition)
{
    loop statements...
}

```

Flowchart:



- While loop starts with the checking of Boolean condition. If it evaluated to true, then the loop body statements are executed otherwise first statement following the loop is executed. For this reason it is also called **Entry control loop**
- Once the condition is evaluated to true, the statements in the loop body are executed. Normally the statements contain an update value for the variable being processed for the next iteration.
- When the condition becomes false, the loop terminates which marks the end of its life cycle.

Example:

```

class main {
    public static void main (String[] args) {
        int i=0;
        while (i<=5)
        {
            System.out.print(i);
            i++;
        }
    }
}

```

Output

012345

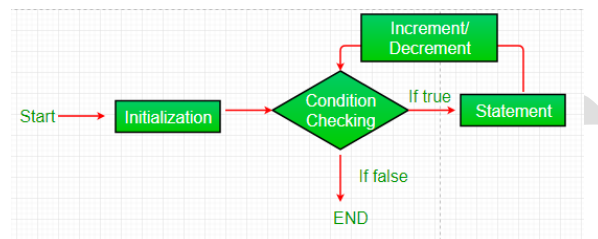
2. for loop:

for loop provides a concise way of writing the loop structure. Unlike a while loop, a for statement consumes the initialization, condition and increment/decrement in one line thereby providing a shorter, easy to debug structure of looping.

Syntax:

```
for (initialization condition; testing condition; increment/decrement)
{
    statement(s)
}
```

Flowchart:



- **Initialization condition:** Here, we initialize the variable in use. It marks the start of a for loop. An already declared variable can be used or a variable can be declared, local to loop only.
- **Testing Condition:** It is used for testing the exit condition for a loop. It must return a boolean value. It is also an **Entry Control Loop** as the condition is checked prior to the execution of the loop statements.
- **Statement execution:** Once the condition is evaluated to true, the statements in the loop body are executed.
- **Increment/ Decrement:** It is used for updating the variable for next iteration.
- **Loop termination:** When the condition becomes false, the loop terminates marking the end of its life cycle.

Example:

```
class main {
    public static void main (String[] args) {
        for (int i=0;i<=5;i++)
        {
            System.out.print(i);
        }
    }
}
```

Output

012345

3. do while:

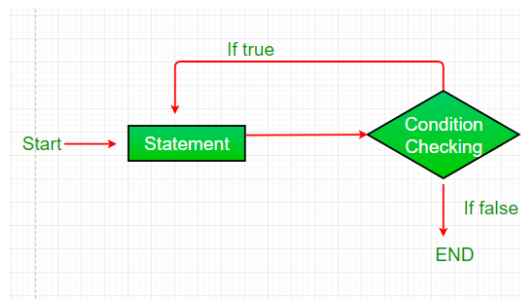
do while loop is similar to while loop with only difference that it checks for condition after executing the statements, and therefore is an example of **Exit Control Loop**.

Syntax:

```
do
{
    statements..
}
```

while (condition);

Flowchart:



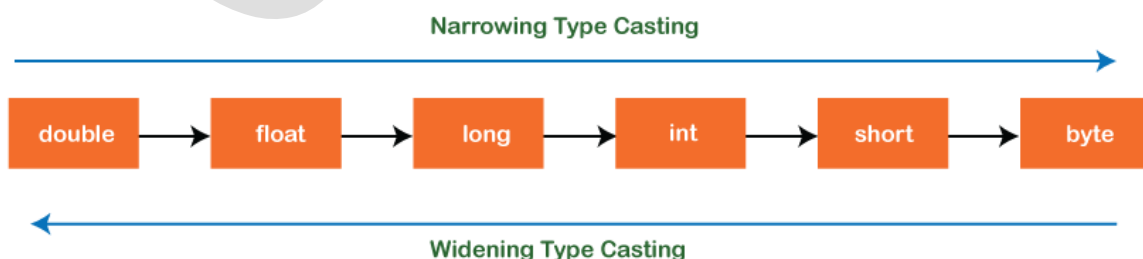
- do while loop starts with the execution of the statement(s). There is no checking of any condition for the first time.
- After the execution of the statements, and update of the variable value, the condition is checked for true or false value. If it is evaluated to true, next iteration of loop starts.
- When the condition becomes false, the loop terminates which marks the end of its life cycle.
- It is important to note that the do-while loop will execute its statements atleast once before any condition is checked, and therefore is an example of exit control loop.

Example:

```
class main {  
    public static void main (String[] args) {  
        int i=0;  
        do  
        {  
            System.out.println(i);  
            i++;  
        }while(i<=10);  
    }  
}
```

Type Casting

type casting is a method or process that converts a data type into another data type in both ways manually and automatically. The automatic conversion is done by the compiler and manual conversion performed by the programmer.



Type Casting in Java

Widening Type Casting

Converting a lower data type into a higher one is called **widening** type casting. It is also known as **implicit conversion** or **casting down**. It is done automatically. It is safe because there is no chance to lose data. It takes place when:

- Both data types must be compatible with each other.
- The target type must be larger than the source type.

byte -> **short** -> **char** -> **int** -> **long** -> **float** -> **double**

Example:

```
int x = 7;  
//automatically converts the integer type into long type  
long y = x;  
//automatically converts the long type into float type  
float z = y;
```

Narrowing Type Casting

Converting a higher data type into a lower one is called **narrowing** type casting. It is also known as **explicit conversion** or **casting up**. It is done manually by the programmer. If we do not perform casting then the compiler reports a compile-time error.

double -> **float** -> **long** -> **int** -> **char** -> **short** -> **byte**

Example:

```
double d = 166.66;  
//converting double data type into long data type  
long l = (long)d;  
//converting long data type into int data type  
int i = (int)l;
```